

Computer Science Department
Software Engineering & Business Analysis

Bachelor's Thesis

MovieCrush

Movie Tracking Platform with Personalized
Recommendations, Detailed Ratings, and Social Features

Anastasiia Syvak

Supervisor

KSE, Denys Zavorodnii, dzavorodnii@kse.org.ua

Expert

KSE, Expert Name, expert@domain.ua

Submission date

16 June 2026

TODOS

1.4 Boxes	11
-----------------	----

- [Personnal todo before marking this thesis as final](#)

Academic Integrity Statement

- I, undersigned, hereby declare that this capstone project is the result of my own work.
- fi All ideas, data, figures and text from other authors have been clearly cited and listed in the bibliography.
 - fi No part of this project has been submitted previously for academic credit in this or any other institution.
 - fi All code, diagrams, and thirdparty materials are either my original work or are used with permission and properly referenced.
 - fi I have not engaged in plagiarism or any form of academic dishonesty.
 - fi Any assistance received -e.g. from peers, tutors, or online forums(is acknowledged in the acknowledgements section.
- I understand that failure to comply with these declarations constitutes academic misconduct and may lead to disciplinary action.

Place, date Kyiv, 0).06.2026

Signature _____

Contents

Acknowledgements	1
Abstract	3
1 Introduction	4
1.1 Basic markup	4
1.2 Images	5
1.3 Tables	5
1.4 Boxes	5
1.5 Citations, Acronyms and Glossary	6
1.6 Code	7
1.7 Context Problem	7
1.8 Objectives	8
1.9 Structure of this report	8
2 Research	9
2.1 Section 1	10
2.2 Section 2	10
2.3 Conclusion	10
3 Design	11
3.1 Section 1	12
3.2 Section 2	12
3.3 Conclusion	12
4 Implementation	13
4.1 Development Sprints	13
4.1.1 Initial Presentation - Discovery Phase	13
4.1.2 Sprint 1 - Month 1: Research and Architecture	13
4.1.3 Sprint 2 - Month 2: MVP	14
4.1.4 Sprint 3 - Month 3: Key Features and AI	16
4.2 UX Design	17
4.2.1 Design Philosophy	17
4.2.2 Visual Identity	17
4.2.3 Screen Overview	18
4.2.4 Design Decisions	22
4.3 Key Algorithms	22
4.3.1 Soulmate Similarity Algorithm	22
4.3.2 Hybrid Recommendation Pipeline	24
4.4 Deployment and CI/CD	25
4.4.1 Deployment Process	25
4.4.2 CI/CD Pipeline	26
5 Validation	28
5.1 Section 1	29
5.2 Section 2	29
5.3 Conclusion	29
6 Conclusion	30

6.1 Project summary	30
6.2 Comparison with the initial objectives	30
6.3 Encountered difficulties	30
6.4 Future perspectives	30
Glossary	31

Acknowledgements

First of all, I want to thank my parents. For giving me the opportunity to study at such a great university, for their support every single time, for always believing in me when I didn't believe in myself - they always knew I would make it. They have always been my biggest role models in life, and everything I have now is because of them. I also want to thank my mom for being someone I can tell absolutely everything to - every secret, every worry, every thought. And I want to thank my dad for teaching me tennis when I was a child - that's what led me to become the head of the KSE Table Tennis Club and find amazing friends at university, who made these student years the best ones.

I want to thank my grandparents - they were always ready to help me with anything, and were even willing to learn how to code just to make things easier for me. Thank you for the fact that when I had classes until 9 PM, I could get to Akademmistechko in 20 minutes instead of an hour and a half, and thank you for always cooking so much and so well every time I came over, as if it was a wedding and not just my last class of the day. I want to thank my brother for always showing up - early or late - even though he wasn't a student here at that moment, just to help or support me.

I want to thank Veronika Anepska for absolutely everything - for always listening to me, for every party, every conversation, every idea, for believing in each other and supporting each other. Together we built discipline - not just opening TikTok every day to keep our streak alive, but also learning Italian every day, doing yoga every day, setting goals and reaching them together. I'm really happy that this diploma was made with you. I'm sure we have an amazing future ahead, because we're doing everything to make it happen.

I want to thank Bohdana Seheda for sitting next to me in an extra English class in first year and Maryna Chernetska for the fact that we were brought together in first year by the same problem - neither of us understood anything. Those small things helped me find such great friends.

Thank you to Ivan Suprun - after talking to you, everything always feels lighter. Thank you to Marko Hlibovytskyi for always telling me I would be fine when I felt like giving up, and for helping me install Windows when I needed it.

I really want to thank the EBD27 and BE27, and my Verkhovyna company. Thank you to Daria Kozlova for the Integration parties, for Halloween vampires, and for inviting me in my fourth year to get through the hard winter moments and the power outages at Hotel Verkhovyna together. Thank you to Dmytro Shevchuk, Matvii Onyshchenko, Artem Litskevych, Dmytro Vasylchuk, Oleksandr Siryi - you are the best. Every holiday, every party, every memory with you is one of the best in my life. Thank you for every tennis game, every good lunch, every conversation, You are all absolute legends. Thank you to Oleh Skakun for always cheering me up and for driving me home.

Thank you to my friend from Italy, Gabriel, whom I met just before the full-scale invasion started - for helping me feel more confident, for every Eurovision night, for respecting my family and my country, and for the invitation to Italy. Thank you also to my friend from Spain, Markel - I'm sure we will meet someday too.

Thank you to Polina Mamchur, Sofiia Tyshchenko, Sofiia Krotova, Yaroslava Mala and Kateryna Shashkina for every moment we went through together, for finding something positive even during the worst deadline weeks, and for always helping.

I also want to thank Kateryna Pyvovarova for our conversation on the 5th floor of KSE - it was you who came up with the Soulmate feature idea, and I'm really glad that conversation happened.

I want to thank the teachers who left a real mark. Thank you to Taras Pavlov - he taught mathematics in the first year and managed to make difficult topics understandable, and more than that - he made me love them even more. Thank you to Yevhen Kubiuk for the practical sessions on algorithms and data structures: I did not miss a single class, because you rarely meet someone who loves what they do so genuinely and wants every student to truly understand. Thank you to Yaroslav Propoi for the practical product analytics course - it was after this course that I understood I want to build my life around analytics. Thank you to Vadym Yaremenko and Ihor Pysmennyi for their huge contribution to my knowledge. Thank you to Volodymyr Skochko - he returned my assignments with new comments every single time, and at some point I genuinely did not know when it would end. But at work I understood what he was preparing me for: clients are often unsatisfied, there are many rounds of revisions, and you just have to keep going. I am ready for that now. And a separate thank you for agreeing to help me install a virtual Windows on my Linux - even though it did not work out, what matters is that he did not say no and tried to help. Thank you to academic director Artem Korotenko - over four years our leadership changed several times, and Artem inherited a situation that was not easy to walk into. He handles it well, and more importantly - you can always come to him with a question and get real help or at least honest advice. That is not something to take for granted. And a very big thank you to my supervisor Denys Zavorodnii. For every meeting, for the time spent reviewing my work, for the guidance, and for walking me through this entire process from start to finish. It was a real pleasure working with you.

Thank you to absolutely everyone who came into my life during these years and made it the best it could be - in such difficult times.

Abstract

The abstract serves as a concise summary of your entire thesis, encapsulating key elements on a single page such as:

- *General background information*
- *Objective(s)*
- *Approach and method*
- *Conclusions*

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1 | Introduction

Your introduction serves to introduce the topic of your Bachelor thesis and to arouse the reader's curiosity with an overview. Why it is important and how it is structured, we explain here.

You can consider an introduction as a teaser for your bachelor thesis. You arouse interest and give a foretaste by presenting your motivation, your method and the state of research in your introduction.

Convince your examiners already in the introduction that your Bachelor thesis will be exciting. If your professor starts reading your thesis with anticipation and interest, the chances of getting good grades are higher.

Pay particular attention to the following in your introduction:

- **Introduce the topic** - What characterizes the topic?
- **Introduce the goal** - What do you want to achieve with your thesis?
- **Make the reader curious** - What motivates the reader to read on?
- **Describe the relevance** - Why is this bachelor thesis scientifically relevant?

The introduction should have the following content:

- **Initial situation presentation of the topic** - You introduce the topic with an exciting 'bait'. You provide initial information on the topic and the object of research and explain the current state of research.
- **Relevance of the topic motivation** - You justify the relevance of your topic (scientifically) and place it in the context of your field. In addition, it is often required that you disclose your personal motivation.
- **Objectives** - Your introduction should clearly state what the goal of your paper is and what outcome you hope to achieve upon completion of the bachelor thesis.
- **Method** - You explain the approach and justify the choice of method.
- **Structure of the Bachelor's thesis** - Finally, you give the reader a general overview of your Bachelor's thesis by explaining the structure, showing the red thread and how the research question is answered.



Welcome to the template's introductory chapter! Instead of boring you with lorem ipsum, here's a quick guide to what you can do in Typst and, more specifically, in this template.

Need more? Check out the [Guide to Typst](#).

1.1 Basic markup

Typst lets you create bold, italic, or monospaced text with ease. You can also sprinkle in equations like $e^{i\pi} + 1 = 0$ or even inline code like `fn main() { println!("Hello, World!") }`. And because life is better in color: pink, blue, yellow, orange, green, and more! **Boldly** colorize!

You can also write numbered or unnumbered lists:

- First item
- Second item
 1. First Subitem

- 2. Second Subitem
- Third item

Need equations? Sure! They look great as blocks too:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}$$

1.2 Images

As they say, a picture is worth a thousand words. Let's add one:



Figure 2: Project logo

1.3 Tables

Tables are great for organizing data. From simple to complex, Typst handles them all:

Name	Age	City
Albert Einstein	25	Bern
Marie Curie	22	Paris
Isaac Newton	30	London

Table 1: Simple table

[31:27]			[24:20]		[19:15]	[14:12]	[11:7]		[6:0]	
funct5		aq	rl	rs2	rs1	funct3	rd	opcode		
5				5	5	3	5	7		

Table 2: Complex table

1.4 Boxes

Highlight key points with these fun boxes (and more):



Infobox: For highlighting information.



Ideabox: Share a brilliant idea.



Warningbox: Proceed with caution!



Firebox: This is 🔥 !



Rocketbox: Shoot for the moon!



Todobox: Just do it!

TODO

Personnal todo before marking this thesis as final

1.5 Citations, Acronyms and Glossary

Add citations with @ like [1] or [1, p.7ff] (stored in **/tail/bibliography.bib**).

Acronym terms like **Infotronics (IT)** expand on first use and abbreviate after **IT**. Glossary items such as **Rust Programming Language (Rust)** can also be used to show their description as such: Rust is a modern systems programming language focused on safety, speed, and concurrency. It prevents common programming errors such as null pointer dereferencing and data races at compile time, making it a preferred choice for performance-critical applications.. Acronyms and glossary entries auto-generate at the document's end (defined in **/tail/glossary.typ**).

1.6 Code

Besides writing inline code as such `fn main() { println!("Hello World") }` you can also write code blocks like this:

```

1 fn main() {
2     let ship = Starship::new("USS Rustacean", (0.0, 0.0, 0.0));
3     let destination = (42.0, 13.0, 7.0);
4     let warp = ship.optimal_warp(ship.distance_to(destination));
5
6     println!("👉 {} traveling to {:?} at Warp {:.2}", ship.name, destination,
7 warp);
8     if warp <= 9.0 {
9         println!("🚀 Warp engaged!");
10    } else {
11        println!("⚠️ Warp failed!");
12    }
13 }
```

Listing 1: First part of the USS-Rustacean code

or directly from a file

```

1 struct Starship {
2     name: String,
3     position: (f64, f64, f64),
4 }
5
6 impl Starship {
7     fn new(name: &str, position: (f64, f64, f64)) -> Self {
8         Self {
9             name: name.into(),
10            position,
11        }
12    }
13    fn distance_to(&self, dest: (f64, f64, f64)) -> f64 {
14        ((dest.0 - self.position.0).powi(2)
15         + (dest.1 - self.position.1).powi(2)
16         + (dest.2 - self.position.2).powi(2))
17        .sqrt()
18    }
19    fn optimal_warp(&self, distance: f64) -> f64 {
20        (distance / 10.0).sqrt().min(9.0)
21    }
22 }
```

Listing 2: Second part of the USS-Rustacean code from `/resources/code/uss-rustacean.rs`

1.7 Context Problem

Haute École d'Ingénierie (HEI) Rust Rust programs

[1], [1, p.7ff]

```

fn main() {
    println!("Hello World!");
}
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1.8 Objectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1.9 Structure of this report

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

2 | Research

In this section, you perform a State of the Art investigation of your domain to understand what's been built, how it works, and where it falls short. You'll survey existing solutions—tools, frameworks and approaches—then compare them against your project's requirements to justify why a new tool (and its specific feature set) is needed.

A focused Domain Area Research unfolds in these steps:

- 1. Clarify your research questions and functional requirements*
- 2. Collect and review candidate solutions*
- 3. Evaluate each alternative's strengths, weaknesses, maturity and architectural style*
- 4. Group similar approaches into meaningful categories (for example, monolithic vs. microservice, commercial vs. open-source)*
- 5. Pinpoint gaps or missing features that motivate your own design*

This process carries your initial concept through systematic research all the way to a clear, actionable set of requirements for your proposed tool.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Contents

2.1 Section 1	10
2.2 Section 2	10
2.3 Conclusion	10

2.1 Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

2.2 Section 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

2.3 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

3 | Design

In this section you turn your requirements into a concrete engineering blueprint. You'll justify every major architectural choice, visualize structure with C4 diagrams for the first three layers, and map out your runtime topology so that peers can understand—and you can defend—every aspect of your system.

1. Clarify how functional and non-functional requirements drive your high-level architecture
2. List each architectural decision (for example, “We chose microservices to enable independent scaling and deployment”) and explain why it best meets your goals
3. Include a C4 Context diagram showing your system in its environment (users, external systems, data sources)
4. Include a C4 Container diagram breaking the system into deployable units (APIs, web front end, background workers, databases) and annotate communication styles and protocols
5. Include a C4 Component diagram for your core container(s), illustrating key modules, services or libraries and their interactions
6. Describe your deployment topology: physical or cloud hosts, network zones, load-balancing, failover and backup strategies
7. Summarize your technology stack, mapping each tool or framework back to a specific container or component and noting any trade-offs (performance, community support, learning curve)
8. Outline how data flows through the system—including storage models, messaging patterns or API contracts—and note any schema or interface versioning plans
9. Address cross-cutting concerns (Security, Logging, Monitoring, Scalability) and show where they sit in your topology

By walking through Requirements → Decisions → Diagrams → Topology, your Design section becomes a rigorous, evidence-backed foundation for the implementation that follows.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Quod idem licet transferre in voluptatem, ut.

Contents

3.1 Section 1	12
3.2 Section 2	12
3.3 Conclusion	12

3.1 Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

3.2 Section 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

3.3 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

4 | Implementation

4.1 Development Sprints

MovieCrush was built over three months in an iterative way, split into three sprints. Before the first sprint, a full discovery phase was completed and presented as a product pitch.

4.1.1 Initial Presentation - Discovery Phase

Before any coding started, a detailed product presentation was prepared. It became the foundation for all decisions that followed. At this stage the following was completed:

- Vision and Mission of the product
- Goals and reasoning for why the idea is promising
- Full market analysis and competitor research
- Description of unique features and how they differ from existing solutions
- Target audience profile and stakeholder analysis
- Business analysis: monetization model, pricing strategy, user growth projections
- Risk assessment and mitigation plan
- First logo ideas
- Design mockups in Figma for five key screens

Deliverables set for Sprint 1:

- Software Architecture Description
- Technology Stack Selection & Justification
- Cost Breakdown & Initial Infrastructure Setup
- Business Process Research Finalization
- MVP implementation

4.1.2 Sprint 1 - Month 1: Research and Architecture

Goals: finish the research phase, design the architecture, and prepare the technical base before coding began.

What was done:

The main outcome of this sprint was a large quantitative study: a survey of 262 respondents distributed through the KSE Slack community. It measured demand for product features, willingness to pay, and current pain points of the target audience. After analyzing the data, charts, and conclusions for each feature - AI recommendations, Wrapped, Soulmate, detailed ratings - Product-Segment Fit was confirmed for the 18-24 age group.

In parallel, data modeling was completed: 26 tables were designed with data types, sizes, and constraints defined, entity relationships and join tables were built (movie_genres, movie_countries). Load was estimated across three growth scenarios, and the 1-9-90 rule was applied to the comments table. Total database size at full load was estimated at 6.4 GB. RPS was also calculated for conservative, realistic, and optimistic scenarios.

An important decision made during this sprint was the approach to AI recommendations. Three services were reviewed - OpenAI, DeepSeek, and Hugging Face Inference API - along with two recommendation strategies: Embedding API + similarity and LLM generation. DeepSeek was selected as the external AI service, and the idea of an in-app AI chat (similar to ChatGPT but limited to film topics) was dropped in favor of a single AI assistant button that generates personalized recommendations.

A preliminary tech stack was defined (Redis for caching, DeepSeek API, AWS for deployment) along with initial architecture diagrams. Both the stack and the diagrams changed significantly during implementation - this is covered in the relevant sections.

Retrospective: No coding was done in this sprint intentionally - the priority was solid research and architecture planning. The number of database tables changed between planning and implementation: from 26 designed to 17 built, as some tables turned out to be unnecessary or were replaced by a different approach (for example, a separate movies table was replaced by the `tmdb_media_cache` strategy).

Deliverables set for Sprint 2:

- Stack setup
- Registration and login
- Movie search
- Adding movies to lists and creating custom lists
- Rating movies and writing comments
- Movie, series, and actor detail pages
- Following other users

4.1.3 Sprint 2 - Month 2: MVP

Goals: build the core of the product - authentication, content catalog, rating system, and main screens.

What was done:

Authentication. A full auth flow was built: registration with validation (username, password, optional first and last name), email confirmation, login, and password recovery via email. Validation runs on the frontend before any request is sent; errors appear under each field individually using a touched state. On the backend, passwords are compared using bcrypt - even if the user does not exist, the comparison still runs against a dummy hash to prevent timing attacks. After a successful login, the `accessToken` and `refreshToken` are saved and navigation fully resets to Home so the user cannot go back.

Home Screen. A header with the MovieCrush logo and a button to go to the user's profile, a footer for tab navigation. Four horizontal content rows: Trending Movies This Week, Trending Series This Week, Upcoming, and Top Rated All Time. Each row is a separate component that takes a title, a data array, and a type (movie or tv) and renders a horizontal FlatList with the first 10 items. Four TMDB requests run in parallel via `Promise.all` when the screen mounts - no repeated requests when navigating back.

Search. Live search with 500ms debounce and a minimum of 2 characters. Three tabs: All, Media, and Cast - switching tabs automatically restarts the search. User search was added in the next sprint.

Movie Screen. Poster, title, IMDb rating, adding to lists (Watched /Watchlist/Favorite/custom), dislike, movie details, trailer link, gallery of film stills, cast with links to actor pages, similar movies. Comments system: signed or anonymous, with or without spoilers, like/dislike, editing and deleting own comments. Rating system: overall score (1-10), detailed score across five criteria, mood after watching, best actor pick, and rating reset. When a movie page opens, four TMDB requests run in parallel: main info, cast, gallery, and similar movies.

Series and Episode Screen. Same as Movie Screen plus season and episode counts. Each season can be expanded to show its description and episode list. Each episode can be opened to view details, be marked as watched, and receive a rating. The series status is shown: ongoing, finished, or cancelled.

Person Screen. Photo, name, place and date of birth, age, biography with read more / read less, filmography with Acting and Crew tabs and links to each title, and known-as names.

Profile Screen. Photo, username, full name, registration date, linked Telegram and Instagram, counters for friends, followers, following, watched movies, series, and episodes. Default and custom lists with the ability to view contents, remove movies, manage privacy, and delete non-default lists.

Settings Screen. Change username, name, and password. Link Instagram and Telegram. Toggle consent for the Soulmate feature. Log out and delete account.

Recommendation Screen. Cold start recommendations using the TMDB /discover endpoint with filters for genre, year, country, and rating. If multiple content types are selected (movies + series), requests run in parallel via Promise.all and results are merged without duplicates. A “New batch” button changes the seed to load the next batch of results.

Data protection. Every request that changes data goes through authMiddleware - without a valid token the server returns 401 Unauthorized. When deleting a list, the server checks not just the list ID but also that it belongs to the current user

Content storage. Movies are not stored in the database - only the tmdb_id. Details are fetched from TMDB on demand.

Retrospective: Following other users was not finished in this sprint and was moved to Sprint 3 as a priority. It also became clear that DeepSeek would not work for AI recommendations - the committee pointed out that the model could ignore a large prompt or return irrelevant results. Finding an alternative became a task for the next sprint.

Deliverables set for Sprint 3:

- Wrapped analytics
- Improved AI recommendations
- Soulmate matching algorithm
- Following users and social features
- Challenges, Instagram share templates, language switch, PDF export, premiere notifications (planned, not implemented due to reprioritization)

4.1.4 Sprint 3 - Month 3: Key Features and AI

Goals: build the features that make MovieCrush different - Soulmate, Wrapped, the social layer, and AI recommendations. Rethink the recommendation engine.

What was done:

Social Features / Follows. A full follower model was built: following and unfollowing other users, viewing their public and default lists, and mutual follow status (friends). When two users are friends, their Instagram or Telegram contacts can optionally be visible to each other. Users can be found through the global search. On movie, series, and episode pages, ratings from followed users are now shown.

Soulmate. A feature that finds the user with the most similar taste. Participation is opt-in - controlled in Settings. The algorithm filters eligible candidates (active accounts, with consent, at least 5 watched movies), computes a similarity score for each one, and saves the best match to the `user_soulmate_matches` table if the score is above the minimum threshold. Recompute is available once per day.

MovieCrush Wrapped. Personalized yearly statistics: total watch time, number of movies, series, and episodes, top genre, top director, top actors, most common mood after watching, typical watch time (cinema ritual), and time spent with the favorite actor. Calculations use cached metadata from `tmdb_media_cache` - no TMDb requests are needed at generation time. Regeneration is available once per day for testing purposes.

AI Recommendations - New approach. During this sprint, a research study was done: 9 models were tested with 5 different prompts each - Claude Opus 4.7, Claude Haiku 4.7, Claude Sonnet 4.6, GPT 5.4, GPT 5.4 mini, GPT 5.5, Gemini Flash, Gemini Pro, and Gemini Thinking. The prompt design was informed by two papers: GroupLens 2024 recommended passing up to 75 movies with ratings and `is_liked/is_disliked` flags, and using a composition rule for result categories (`strong_match`, `diversity`, `hidden_gem`). NEC 2025 showed that prompt structure matters more than model choice, and that cheaper models like Gemini Flash give comparable quality at 90% lower cost.

The full model comparison, prompt designs, and per-model outputs are documented at: [AI Recommendations Research - Notion](#).

However, the committee raised concerns about a purely LLM-based approach: the model couldn't return art-house picks, get lost in a long prompt, or miss context when given many movies. This led to the decision to move to a hybrid pipeline with collaborative filtering.

CF Service - New recommendation engine. A separate Python microservice on FastAPI was built with an ALS model from the implicit library. The model is trained on a user-item interaction matrix (views, ratings, favorites) with BM25 weighting. Gemini stayed in the final architecture but in a different role - not as the main recommendation source, but as a re-ranker: it receives candidates from ALS and TMDb Discover and picks the 15 best ones with a category label (`strong_match`, `diversity`, `hidden_gem`).

For new users without enough watch history, an Onboarding flow was added: users pick favorite actors and movies at the start, and initial recommendations are served through TMDb Discover based on those signals until enough data is collected for ALS.

Retrospective: Several planned deliverables were intentionally not built - after reviewing survey results, the top priorities were clear: Wrapped (rated 4.24/5), AI recommendations, and social features. Not implemented: Challenges, Instagram share templates, language switch, PDF export, and premiere push notifications. These are planned for future development.

4.2 UX Design

4.2.1 Design Philosophy

MovieCrush uses a dark theme as the only design option. This was a deliberate choice - film posters are colorful by nature, and a dark background makes them stand out without competing visual noise. All screens share the same color palette, typography, and spacing system, which creates a consistent feel as the user moves through the app.

4.2.2 Visual Identity

Color palette. The palette uses four main colors: pure black (**#000000**) as the background, gold (**#FFD700**) as the primary action color, soft pink (**#FFAFCC**) as the accent used for the logo and headings, and white (**#FFFFFF**) for body text. Card surfaces use **#111111** and **#1A1A1A** to create subtle depth without breaking the dark theme. Error states use red (**#FF4D4D**) and success states use green (**#00CC66**).

Typography. The entire app uses Poppins - a geometric sans-serif font with four weights: Regular, Medium, SemiBold, and Bold. Poppins was chosen for its clean look on small screens and its good readability at both large display sizes and small caption sizes. All font constants are centralized in a single file so that changing a weight applies consistently across the whole app.

Logo. The MovieCrush logo combines the letters M and C using the brand's gold-to-pink gradient. The C is styled to suggest a heart shape, reinforcing the "crush" concept. The icon was designed to hold up at small sizes - as seen on the home screen and in the app drawer alongside Netflix and Spotify.

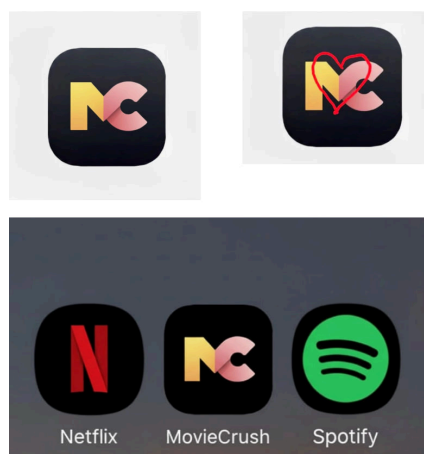


Figure 3: MovieCrush app icon

4.2.3 Screen Overview

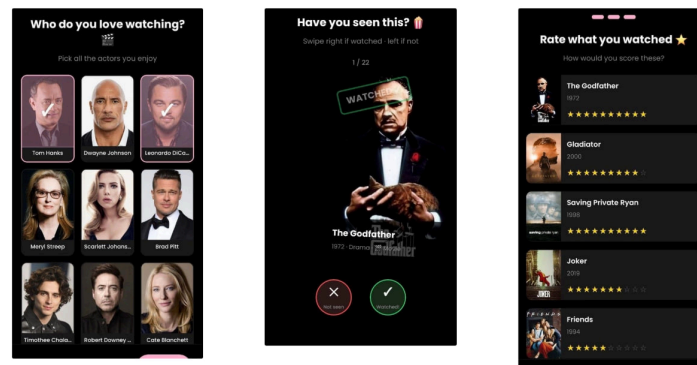


Figure 4: Onboarding - actor selection, movie swipe cards, and optional ratings

The Onboarding flow runs once when a new user signs up and serves as the foundation for cold start recommendations. It has three steps. First, the user picks favorite actors from a curated grid. Second, a swipe card interface shows popular movies, series, anime, and cartoons one by one - the user swipes right if they have seen it or left if not, or uses the buttons below. If a user has not seen anything in the first batch, a second batch of 20 different titles is shown automatically to ensure at least some signal is collected. Third, for every title marked as watched, the user can optionally leave a star rating. All of this data feeds directly into the cold start recommendation engine.

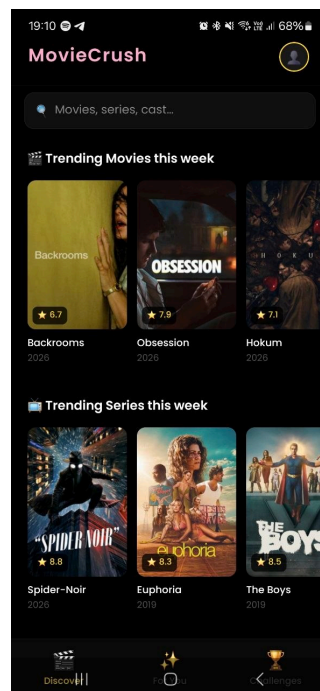


Figure 5: Home Screen - trending movies and series, search bar, bottom navigation

The Home Screen is the entry point of the app. It shows four horizontal content rows fetched from TMDB: Trending Movies This Week, Trending Series This Week, Upcoming, and Top Rated All Time. The MovieCrush logo in pink sits in the top left, with a profile

avatar button on the right. The bottom navigation bar has three tabs: Discover, For You, and Challenges.

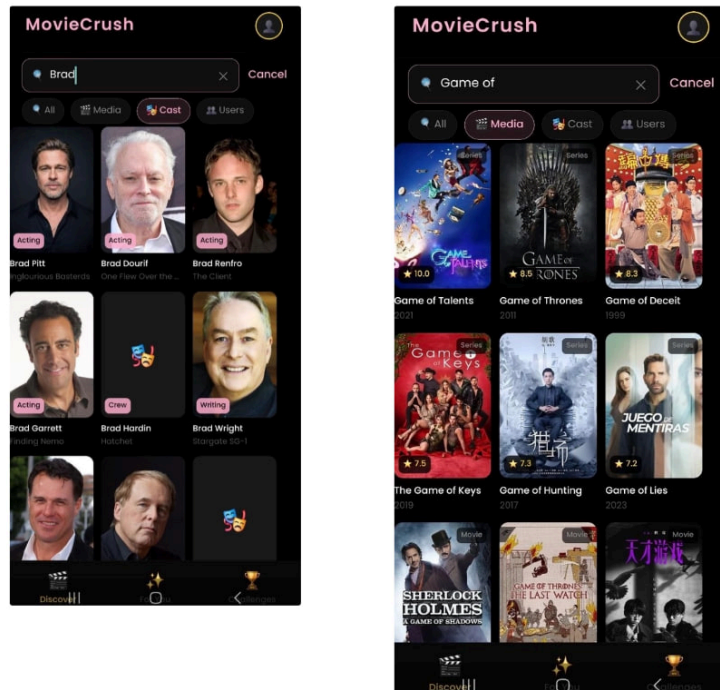


Figure 6: Search Screen - live search with All, Media, Cast, and Users tabs

The Search Screen activates from the search bar on the Home Screen. Results are shown in a 3-column poster grid with ratings and year. Four filter tabs let the user switch between All, Media, Cast, and Users results.

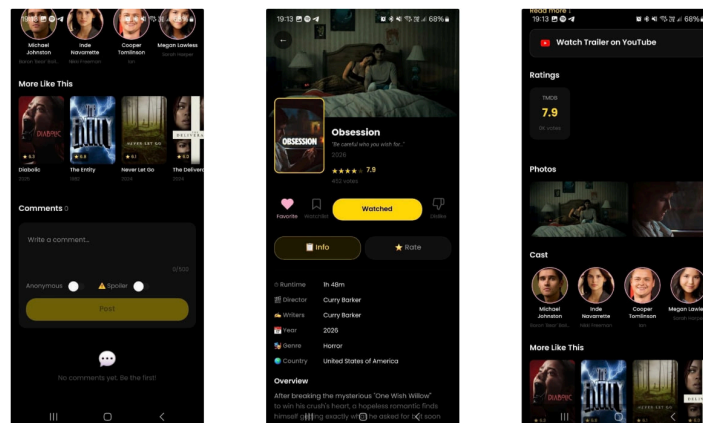


Figure 7: Movie Screen - hero backdrop, action bar, info tab, cast, comments

The Movie Screen opens with a full-width backdrop image at the top, with the poster overlaid in the bottom-left corner. Below the hero are four quick actions: Favorite, Watchlist, a large Watched button, and Dislike. Two tabs - Info and Rate - split the movie details from the rating interface. The Info tab shows runtime, director, writers, year, genre, country, overview, trailer link, gallery, cast with links to actor pages, similar movies, and

a comments section. Comments can be signed or anonymous, with or without a spoiler warning.

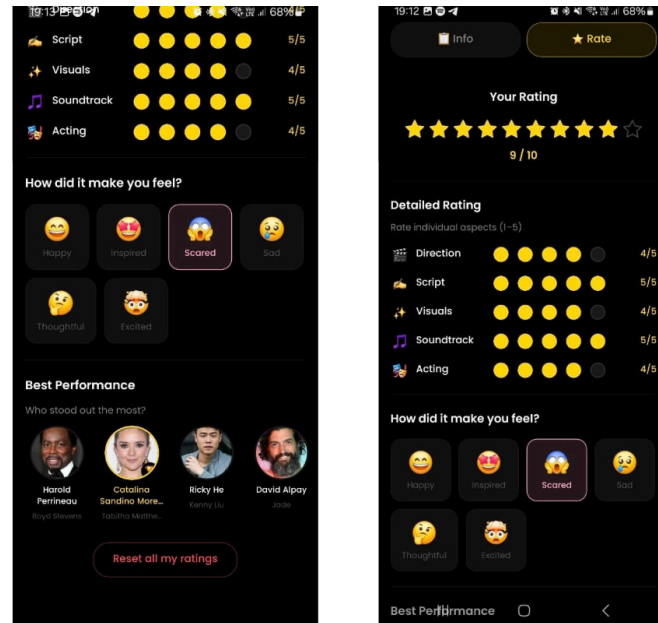


Figure 8: Rate tab - star rating, detailed 5-criteria score, mood picker, and best performance

The Rate tab shows a 10-star rating input, a detailed breakdown across five criteria (Direction, Script, Visuals, Soundtrack, Acting - each rated 1-5), a mood picker with six options (Happy, Inspired, Scared, Sad, Thoughtful, Excited), and a Best Performance section where the user votes for the standout actor.

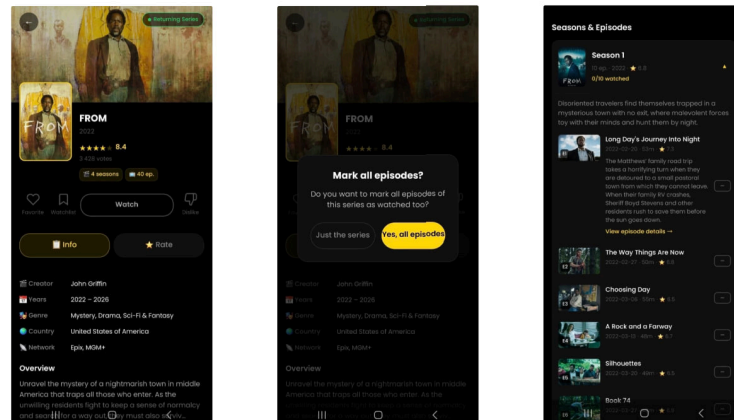


Figure 9: Series Screen - series info, mark all episodes dialog, and season episode list

The Series Screen follows the same structure as the Movie Screen with additional season and episode tracking. When marking a series as watched, a dialog asks whether to mark just the series or all individual episodes as well. Each season can be expanded to show individual episodes with their air date, runtime, TMDB rating, and a short description. Each episode can be marked as watched separately. An episode page shows details and allows the user to leave a rating just for that episode.

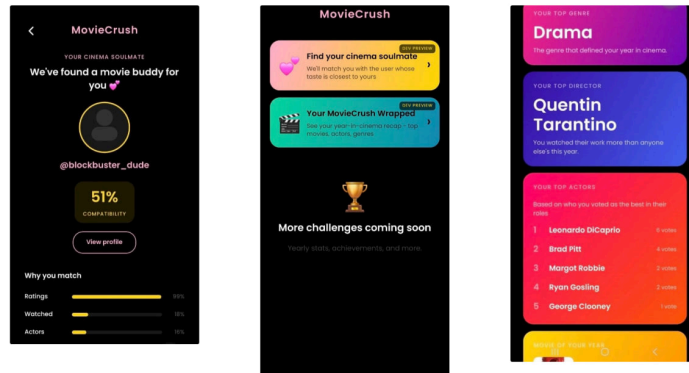


Figure 10: Challenges Screen, Soulmate result, and Wrapped yearly stats

The Challenges Screen is the entry point for two signature features. The Soulmate card launches the matching algorithm and shows the result: the matched user's profile, a compatibility percentage, and a breakdown of why they match - with gold progress bars for Ratings, Watched, and Actors similarity. The Wrapped feature presents a series of full-screen gradient slides inspired by Spotify Wrapped, each focusing on one personal stat: top genre, top director, top actors with vote counts, and more.

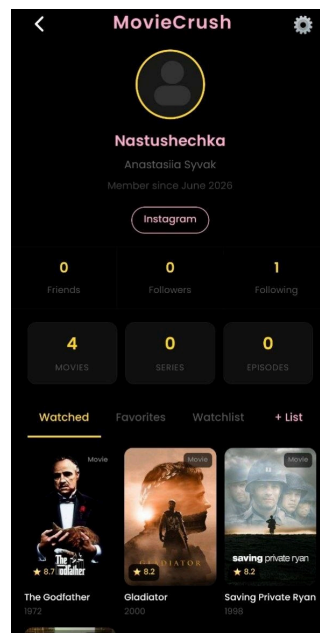


Figure 11: Profile Screen - avatar, social stats, content counters, and movie lists

The Profile Screen shows the user's avatar with a gold ring border, username in pink, full name, membership date, and optional Instagram button. Three counters show social stats (Friends, Followers, Following) and three more show content stats (Movies, Series, Episodes watched). Below are the user's lists in a scrollable tab layout: Watched, Favorites, Watchlist, and custom lists.

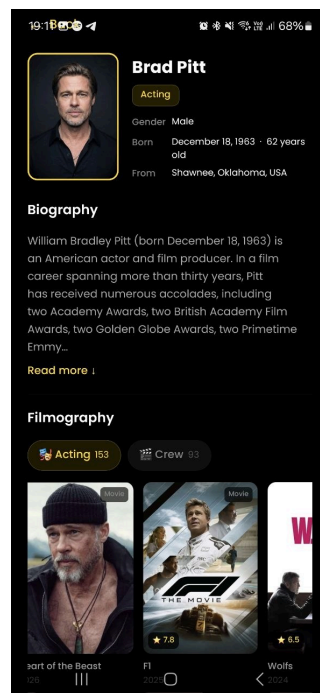


Figure 12: Actor Screen - bio, Acting and Crew filmography tabs

The Actor Screen shows a gold-bordered portrait photo, key facts (gender, birth date, birthplace, age), a biography, and a filmography split into Acting and Crew tabs. Each filmography item is a poster card with a rating badge and a link to the title's page.

4.2.4 Design Decisions

Figma mockups. Initial screen layouts were designed in Figma before coding began. The mockups covered the five core screens. During implementation some layouts changed - new ideas came up and certain components were improved beyond the original design. The Figma file is available at: [MovieCrush Figma](#).

Logo concept. The logo ideas are documented at: [Logo Ideas - Notion](#).

Dark theme only. A light theme was considered but dropped. The app is used primarily in evening and low-light settings, and movie poster art looks significantly better on a dark background.

Emoji as visual language. Section headers and filter chips use emoji alongside text labels. This was a deliberate choice to keep the interface feeling friendly and expressive rather than corporate - consistent with the target audience of 18-24 year olds.

4.3 Key Algorithms

4.3.1 Soulmate Similarity Algorithm

The Soulmate feature finds the user whose film taste is closest to yours. The algorithm computes a similarity score between two users based on five independent metrics, combines them into a single weighted score, and saves the best match if it passes a minimum threshold.

4.3.1.1 Five Similarity Metrics

Each metric measures a different dimension of taste overlap and uses a different mathematical technique suited to the nature of the data.

Metric 1 - Rating Cosine Similarity (weight: 0.45). This is the strongest signal. For every movie both users have rated, the algorithm builds two parallel numeric vectors and computes cosine similarity between them. The key insight is that cosine similarity measures the angle between vectors rather than their magnitude - so a user who consistently rates 9/10 and a user who rates 7/10 for the same films can still show high similarity if their relative preferences match. A minimum overlap of 3 jointly rated movies is required before this metric is computed - below that threshold, the score defaults to 0.

Metric 2 - Watched Overlap / Jaccard (weight: 0.22). Measures what share of total watched titles both users have in common. Jaccard similarity divides the size of the intersection by the size of the union of both watch lists, so the score naturally accounts for users with very different library sizes - a user with 10 watched movies and a user with 200 are compared fairly.

Metric 3 - Actor Overlap / Jaccard (weight: 0.16). Each time a user rates a movie, they can vote for the best actor in that film. This metric computes Jaccard similarity between the two users' sets of voted actors - capturing a deeper layer of taste alignment that goes beyond genre preferences.

Metric 4 - Mood Cosine Similarity (weight: 0.11). After watching a film, users select a mood: Happy, Inspired, Scared, Sad, Thoughtful, or Excited. Each user's mood history is treated as a frequency vector - how many times each mood was selected - and cosine similarity is computed across the shared mood space. Two users who consistently feel the same emotions after watching the same types of films are considered more compatible.

Metric 5 - Disliked Overlap / Jaccard (weight: 0.06). Shared dislikes are a weak but real signal - if two users consistently dislike the same films, their tastes are likely to align in the opposite direction as well. Jaccard similarity is computed on each user's set of disliked titles.

4.3.1.2 Weighted Score

The five metrics are combined into a single final score using fixed weights that sum to exactly 1.0. Rating carries the highest weight because numeric ratings are the most explicit and precise signal a user provides. Watched overlap is second - a shared viewing history is a strong implicit signal even without explicit ratings. Actor preferences and mood patterns follow as secondary signals. Shared dislikes have the lowest weight: agreement on what to avoid is meaningful but less decisive than positive preference alignment.

Metric	Method	Weight
Rating similarity	Cosine similarity	0.45
Watched overlap	Jaccard similarity	0.22
Actor overlap	Jaccard similarity	0.16
Mood similarity	Cosine similarity (bag of counts)	0.11
Disliked overlap	Jaccard similarity	0.06

Table 3: Soulmate similarity metrics and their weights

4.3.1.3 Full Search Flow

The algorithm first filters eligible candidates - active users who have given consent to the feature and have watched at least 5 movies. For each candidate, all five metrics are computed in parallel to minimize latency, and the weighted score is calculated. The algorithm keeps track of the highest-scoring candidate across all comparisons.

A match is saved only if the final score exceeds the minimum threshold of 0.30 - below this value the algorithm returns no match rather than suggesting a low-quality result. The result is stored in the `user_soulmate_matches` table with the full metric breakdown, so the UI can show the user exactly why they were matched - for example, Ratings 99%, Watched 18%, Actors 16%. Recompute is limited to once per 24 hours.

4.3.2 Hybrid Recommendation Pipeline

The recommendation system is a three-stage pipeline: collaborative filtering generates candidates, TMDB Discover adds diversity, and Gemini re-ranks the combined pool into a final list.

4.3.2.1 Stage 1 - ALS Collaborative Filtering (Python CF Service)

The CF service is a separate Python microservice on FastAPI. It trains an ALS (Alternating Least Squares) model from the `implicit` library on a user-item interaction matrix built directly from the PostgreSQL database.

Interaction matrix construction. Each user-movie interaction is converted to a preference score that reflects the strength of the signal. Adding a movie to favorites scores highest (5.0), followed by high ratings (4.0 for 8+, 3.0 for 6-7, 2.0 for 4-5), then watchlist additions (1.5), and finally movies that were only marked as watched without a rating (1.0). The resulting matrix is stored in scipy CSR (Compressed Sparse Row) format - since most user-item pairs have no interaction at all, a sparse representation keeps memory usage low.

BM25 weighting. Before training, BM25 weighting is applied to the matrix. BM25 is a technique from information retrieval that normalizes interaction counts - it prevents users with very large watch histories from dominating the model's learned representations and ensures that less active users still contribute meaningfully to the training signal.

Model training. The ALS model learns latent factor representations for both users and items by alternately solving for user factors and item factors while keeping the other fixed - this is the "alternating" part of ALS. The model parameters (factors=24, iterations=30,

regularization=1.0, alpha=20.0) were selected through a grid search optimizing Hit Rate10 and MRR (Mean Reciprocal Rank). The trained model and index mappings are saved to disk so that inference does not require retraining on every request.

Inference. At request time, the model computes a score for every item the user has not yet interacted with and returns the top N candidates as TMDB IDs. Already watched and disliked titles are filtered out before returning results.

4.3.2.2 Stage 2 - TMDB Discover Candidates

ALS only recommends items that were present in the training matrix. To add variety and cover cases where a user's taste is unusual or their history is small, a second candidate source is built from the TMDB Discover API. The system builds a user profile from watch history - top 3 genre IDs, top voted actor, and allowed content buckets (movie, tv, anime, dorama, animation) - and fires parallel Discover requests for each content type the user has shown interest in. Results are deduplicated and merged into a single candidate pool, each item tagged with its source (ALS or Discover).

4.3.2.3 Stage 3 - Gemini Re-ranking

The tagged candidate pool is sent to Google Gemini 2.5 Flash along with a sample of the user's watch history and ratings. Gemini selects the 15 best titles from the pool and assigns each a category: **strong_match** (clearly fits the user's established taste), **diversity** (different from usual but plausibly enjoyable), or **hidden_gem** (lesser-known or under-rated title worth discovering). Results are cached in the **user_ai_recommendations** table with a 24-hour TTL to avoid calling the model on every request. If Gemini fails, the system falls back to returning the ALS-priority candidates without re-ranking.

4.3.2.4 Cold Start Strategy

New users do not yet have enough watch history for ALS to produce meaningful results. For these users, a dedicated cold start service runs instead. During onboarding, users select favorite actors and rate a curated set of movies. The cold start service analyzes these signals: genre weights are computed per movie with higher-rated movies contributing more weight (rating 8+ contributes weight 3, rating 6-7 contributes weight 2), content buckets are inferred from the types of content the user responded positively to, and three parallel TMDB Discover queries run - by actor, by genre, and by popularity. Results from all three sources are merged using an interleaving strategy to keep the content type distribution balanced across the final batch of 25 recommendations. Once a user accumulates enough watch history, the system automatically switches to the full ALS + Gemini pipeline on the next recommendation request.

4.4 Deployment and CI/CD

4.4.1 Deployment Process

4.4.1.1 Infrastructure Decisions

Choosing where to host MovieCrush went through two iterations before landing on the final setup.

AWS (dropped). The original plan was to deploy on AWS. However, the AWS account had previously been blocked due to an outstanding balance. The decision was made not to restore it and look for an alternative - the risk of getting blocked again in the middle of active development was too high.

Render + Neon (intermediate). The next setup was Render for the backend and Neon as a separate managed PostgreSQL provider. This worked but had two real problems. First, the database was outside Render's network, which meant every query from the backend had to travel over the public internet to reach it. This added latency to every single request. Second, having the database reachable from the public internet is a security concern in itself - the database was not inside a private network with the backend, so it was exposed in a way that a production system should avoid.

Render only (final). Moving both the backend and the database to Render solved both problems. Render Postgres is managed PostgreSQL hosted in the same infrastructure as the backend services. Queries between the backend and the database stay inside Render's private network - no public internet hop, lower latency, and the database is not exposed outside the private network. Render also provides automatic daily backups and handles PostgreSQL maintenance automatically.

4.4.1.2 Docker Multi-stage Build

Both the backend and the CF service are packaged as Docker images using multi-stage builds. The multi-stage approach keeps the final image small: a build stage compiles TypeScript or installs all Python dependencies, and a separate runtime stage copies only what is needed to run.

For the backend, the build stage installs all dependencies including dev tools and compiles TypeScript to **dist/**. The runtime stage starts fresh, installs only production dependencies, and copies just the compiled output. This means the final image does not carry TypeScript, ts-node, or any other build-time tooling.

Both services are deployed on Render as separate web services. They can be updated and redeployed independently - for example, retraining the ALS model and redeploying the CF service does not require touching the backend.

4.4.1.3 Mobile Distribution

The mobile app is built using EAS Build (Expo Application Services). Building in the cloud removes the need for a local Mac to produce the iOS-compatible bundle. For Android testing and distribution, EAS produces an APK that can be shared directly - no Play Store account is required for this stage.

4.4.2 CI/CD Pipeline

The CI pipeline runs on GitHub Actions on every push to **main** and on feature, fix, and chore branches, as well as on pull requests to **main**. It has four jobs that run in a defined order.

Job 1 - Backend: TypeScript + tests. Installs Node.js 20, runs **npm ci**, then performs three checks in sequence: TypeScript type check, the Jest unit test suite, and a production build. All three must pass before the Docker build job is allowed to start.

Job 2 - Mobile: TypeScript check. Installs dependencies for the React Native project and runs a TypeScript type check. This catches type errors in the mobile codebase without needing to run a full Expo build on every push.

Job 3 - CF Service: install + import check. Sets up Python 3.11, installs requirements, and runs a quick import check to confirm that the core modules load without errors. This catches broken dependencies or import-time failures before a broken CF service image gets built.

Job 4 - Docker: build images. Runs only after the backend and CF service checks pass. Builds both Docker images to confirm that the full containerized build works end to end. Deployment to Render is triggered automatically on every commit. Render detects the new commit and redeploys the service without any manual steps - so a passing CI run and a successful deploy happen together as part of the same push.

5 | Validation

Validation (Requirements Verification and Testing)

In this section you demonstrate how your implementation satisfies the initial requirements through clear testing methods and concise examples—suitable for a bachelor-level project:

1. *Restate each key functional and non-functional requirement from your Analysis and Design sections*
2. *Describe the testing approach for each requirement (for example, unit tests, manual acceptance checks or scenario walkthroughs)*
3. *Provide concrete test cases or usage examples that show how you verify each requirement in practice*
4. *Summarize actual versus expected outcomes, indicating pass/fail status for each test*
5. *Include brief snippets of test code or sample console outputs to illustrate your procedures*
6. *Note any gaps or deviations and suggest simple fixes or areas for future improvement*
7. *If a feature wasn't intended for specific scenarios (e.g. high-load), omit unrealistic stress tests and clearly document its current limitations*

This focused structure ties every requirement directly to validation results, using examples and methods you can realistically carry out at the bachelor level.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Quod idem licet transferre in voluptatem, ut.

Contents

5.1 Section 1	29
5.2 Section 2	29
5.3 Conclusion	29

5.1 Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

5.2 Section 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

5.3 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

6 | Conclusion

Conclusion

In this final section you bring together your work and reflect on its impact. Keep it concise, restating key points without introducing new information:

- 1. Project Summary: Briefly recap objectives, methodology and principal results*
- 2. Alignment with Objectives: Discuss how outcomes meet initial goals, referencing requirements and design aims*
- 3. Lessons Learned and Challenges: Note any obstacles and how they informed improvements*
- 4. Limitations: Acknowledge features or scenarios beyond this scope and clearly state current system boundaries*
- 5. Future Work: Suggest practical enhancements or research directions building on your findings*

Avoid introducing new concepts here; refer readers to the Discussion for deeper analysis.

6.1 Project summary

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam quisque id diam vel quam elementum ac venenatis. Cras mattis consectetur purus sit amet fermentum. Aenean lacinia bibendum nulla sed consectetur. Quod idem licet transferre in voluptatem, ut.

6.2 Comparison with the initial objectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam quisque id diam vel quam elementum ac venenatis. Cras mattis consectetur purus sit amet fermentum. Aenean lacinia bibendum nulla sed consectetur. Quod idem licet transferre in voluptatem, ut.

6.3 Encountered difficulties

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam quisque id diam vel quam elementum ac venenatis. Cras mattis consectetur purus sit amet fermentum. Aenean lacinia bibendum nulla sed consectetur. Quod idem licet transferre in voluptatem, ut.

6.4 Future perspectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam quisque id diam vel quam elementum ac venenatis. Cras mattis consectetur purus sit amet fermentum. Aenean lacinia bibendum nulla sed consectetur. Quod idem licet transferre in voluptatem, ut.

Glossary

Programming Language

Rust – Rust Programming Language: Rust is a modern systems programming language focused on safety, speed, and concurrency. It prevents common programming errors such as null pointer dereferencing and data races at compile time, making it a preferred choice for performance-critical applications. [6](#), [7](#)

University

HEI – Haute École d'Ingénierie [7](#)

IT – Infotronics [6](#)

Bibliography

- [1] S. Zahno *et al.*, “Dynamic Project Planning with Digital Twin,” *Frontiers in Manufacturing Technology*, vol. 3, May 2023, doi: [10.3389/fmtec.2023.1009633](https://doi.org/10.3389/fmtec.2023.1009633).

A | Appendix

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.